# CSE 151A - Homework 9

due Thursday, June 5 at 11:59PM

**Instructions:**

- You may discuss problems with your study group, but ultimately all your work (mathematical problems, code, experimental details) must be individual.
- Your solutions must be `typed up` and uploaded to Gradescope by 11:59PM on Thursday, June 5. No late homeworks will be accepted under any circumstances, so you are encouraged to upload early.
- A subset of the problems will be graded.

## Conceptual and mathematical problems

**Problem 1.** *Neural net parametrization*

A feedforward neural net has an input layer with 10 nodes, followed by four hidden layers with 1000 nodes each, and an output layer with one node. Each layer is fully connected to the previous layer. What is the total number of parameters, approximately? You may disregard the offset term at each node (the "b" in the linear function) if you like.

**Solution:** There are $10 * 1000 + 3 * 1000 * 1000 + 1000 = 3011000$ parameters.

**Problem 2.** *Softmax probabilities*

The output layer of a particular neural net has four nodes $y_1, y_2, y_3, y_4$, representing four labels in a classification problem. For some input $x$, these nodes end up with the values

$$y_1 = 1.0, y_2 = 0.0, y_3 = -1.0, y_4 = 0.0$$

and these are converted into probabilities using a softmax. What is the most likely label, and what probability is assigned to it?

**Solution:** The most likely label is 1, and the probability is $e^1/(e^1 + e^0 + e^{-1} + e^0) = 0.5344$, or 53.44%.

**Problem 3.** *Exclusive-OR*

The XOR Boolean function, $\{0,1\}^2 \to \{0,1\}$, has the following behavior:

| $x_1$ | $x_2$ | XOR$(x_1, x_2)$ |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

Show how to implement this function using a neural net with two input units $(x_1, x_2)$, an output unit, and two hidden units with ReLU activation.

**Solution:** The neural net is $h_1 = \text{ReLU}(x_1 + x_2 - 1)$, $h_2 = \text{ReLU}(1 - x_1 - x_2)$, and $y = 1 - h_1 - h_2$.

**Problem 4.** *Decipher the net*

**a)** Which function $\{0,1\}^2 \to \{0,1\}$ is realized by the following neural net with two input units $x_1, x_2$, two hidden units $h_1, h_2$, and one output unit $y$?

- $h_1 = \text{ReLU}(x_1 + x_2)$
- $h_2 = \text{ReLU}(x_1 + x_2 - 1)$
- $y = h_1 - h_2$

> **Solution:** This is the OR function since if $x_1 = x_2 = 0$, then $y = \text{ReLU}(0) - \text{ReLU}(-1) = 0$ and otherwise, $y = (x_1 + x_2) - (x_1 + x_2 - 1) = 1$.

**b)** Which function $\mathbb{R} \to \mathbb{R}$ is realized by the following neural net with one input unit $x$, two hidden units $h_1, h_2$, and one output unit $y$?

- $h_1 = \text{ReLU}(x)$
- $h_2 = \text{ReLU}(-x)$
- $y = h_1 + h_2$

> **Solution:** This is the absolute value function since $y = \max(x, -x) = |x|$.

## Programming problems

Before undertaking these problems, install PyTorch on your computer and download the archive files `simple-nn.zip` and `hymenoptera.zip` from the course website.

**Programming Problem 5.** *Credit card fraud data*

Download the data set at https://www.kaggle.com/mlg-ulb/creditcardfraud. This data set has details of 284,807 credit card transactions, some of which are fraudulent. Each transaction is represented by 28 features (scrambled using PCA as a primitive kind of anonymization), and has a corresponding label (1 is fraudulent and 0 is legitimate).

**a)** How many of the transactions are fraudulent? Why might this be problematic when learning a classifier?

> **Solution:** There are 492 fraudulent transactions, or 0.17%. This is problematic because the data set is highly imbalanced, and the classifier will likely be biased towards the majority class.

**b)** Downsample the legitimate transactions to make the data set more balanced. For instance, you might try to make sure that each class (legitimate or fraudulent) makes up at least 25% of the data set.

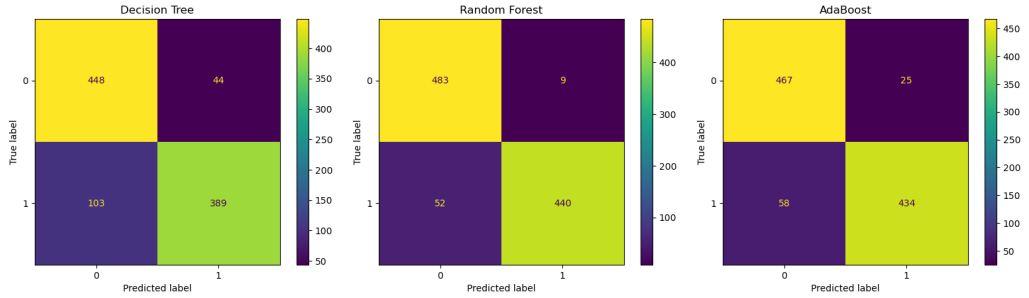
```
num_fraud = df[df['Class'] == 1].shape[0]
legit = df[df['Class'] == 0].sample(n=num_fraud)
fraud = df[df['Class'] == 1]
balanced = pd.concat([fraud, legit])
```

**c)** Fit three kinds of classifier to the data:

- decision tree
- boosted decision stumps
- random forest

In each case, use cross-validation to estimate the confusion matrix. For doing the cross-validation, you might find it helpful to use `sklearn.model_selection.cross_val_predict`. For computing the confusion matrix, you can use `sklearn.metrics.confusion_matrix`. If you want a slick display of the confusion matrix, you might try `sklearn.metrics.ConfusionMatrixDisplay`.

**Solution:**

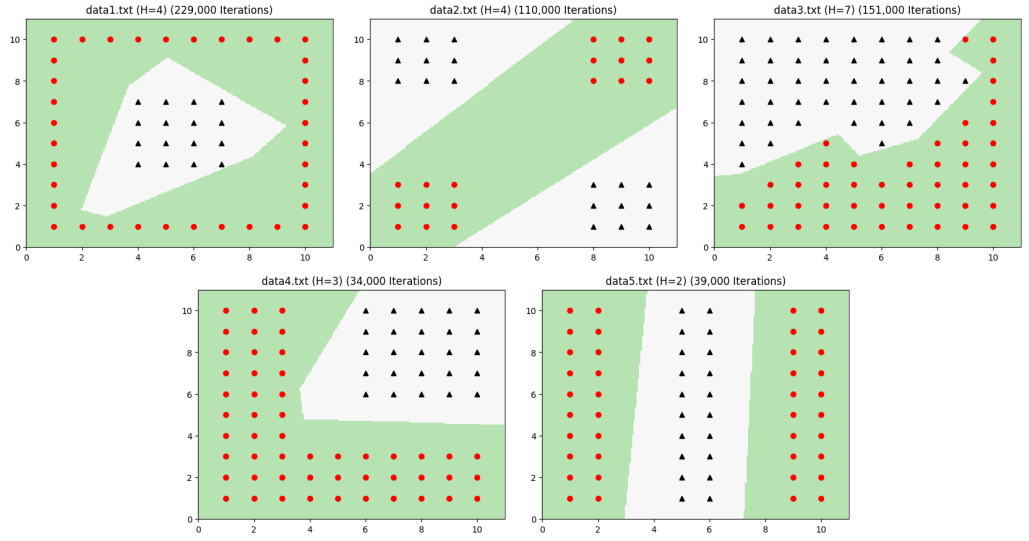



**Programming Problem 6.** *Neural net experiments*

When you unzip `simple-nn.zip`, you will see a Jupyter notebook called `simple-nn.ipynb`. Look through it and run the cells in Sections 1 and 2. These load in a data set called `data1.txt`, train a neural net on it, and print the decision boundary.

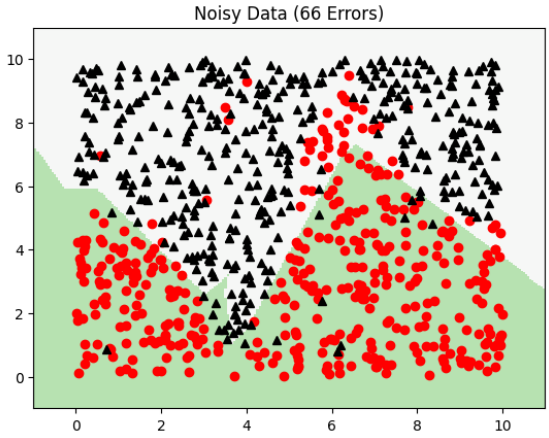
**a)** Look at the training code. You will see that the neural net has **one** hidden layer with $H$ nodes. Train the net on the five data sets `data1.txt` through `data5.txt`. For each, try two values of $H$, and train multiple times (at least five), picking the lowest-error solution. For each of the data sets, plot the best model you found, state what value of $H$ produced it, and state the number of iterations till convergence.

**Solution:**



**b)** Now move to Section 3 of the notebook. This generates a new data set that is highly noisy. To deal with it, copy over the neural net code from above and modify it to train a net with **two** hidden layers, each with $H$ nodes. Train the net a bunch of times and print the best boundary you find. You should aim for $< 100$ errors on the training set (this should be possible even with $H = 4$).

**Solution:**




**Programming Problem 7.** *A computer vision classification task*

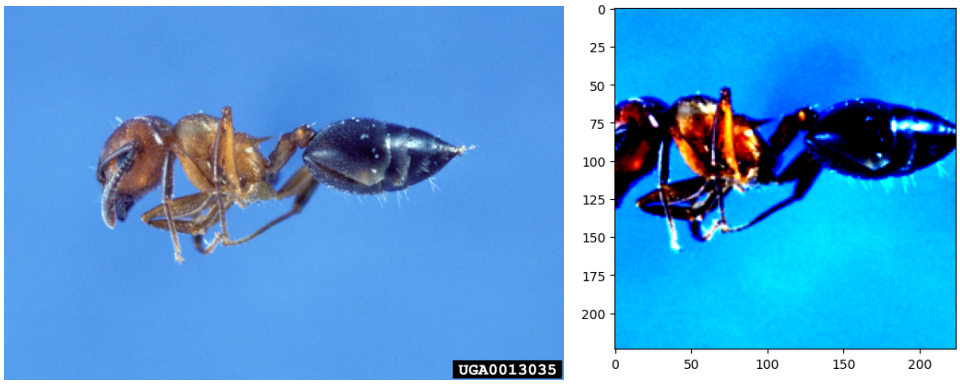
In this problem, we'll train a classifier that takes as input an image of an ant or a bee, and determines which it is.

When you unzip `hymenoptera.zip` you will see a Jupyter notebook, called `ants-bees.ipynb` and a directory called `hymenoptera_data`.

**a)** The `hymenoptera_data` directory has a subdirectory with the training set (`train`) and another with the test set (`val`). Take a look at some of the images. They are of varying sizes, with diverse backgrounds. And there are only 244 training images: not an easy learning problem!

Run the first few cells of the notebook to see how the images are normalized in size. Pick an image from the directory and show both the original version and the normalized version.

**Solution:**



**b)** Run the remaining cells of the notebook to see how the ResNet50 network is used to produce a 2048-dimensional representation for each image, and how a logistic regression classifier is constructed on top of this. Report the test accuracy of the logistic regression classifier.

**Solution:** The test accuracy is 80.39%.

**c)** Now, use this same 2048-d representation to construct a k-nearest neighbor classifier. Give the test accuracies obtained for $k = 1, 3, 5$. Note: If you use `sklearn.neighbors.KNeighborsClassifier`, you might want to set `algorithm='kd tree'`.

**Solution:**

| k | Test Accuracy |
|---|---|
| 1 | 69.28% |
| 3 | 71.24% |
| 5 | 69.28% |